

Algoritmi e Strutture Dati – Parte A – 14/07/25

Esercizio -1 Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

A1 – Complessità – Punti ≥ 8

Trovare limiti superiori e inferiori per la seguente equazione di ricorrenza, utilizzando il metodo di sostituzione (detto anche per tentativi)

$$T(n) = \begin{cases} T\left(\left\lfloor \frac{n}{\sqrt{2}} \right\rfloor\right) + 2T\left(\left\lfloor \frac{n}{\sqrt{5}} \right\rfloor\right) + n^2 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

A2 – Albero bello – Punti ≥ 10

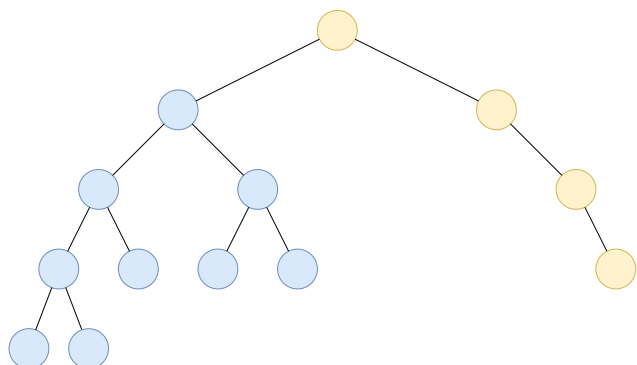
Come forse sapete, un *alberobello* è un albero binario in cui tutti i nodi hanno zero o due figli, e nessun nodo ha un solo figlio. Anche se è difficile che un albero sia un alberobello, molti dei suoi sottoalberi sono alberibelli. Il vostro compito è scrivere un algoritmo

```
int largestAB(TREE T)
```

che prenda in input un albero e restituisca la dimensione del più grande sottoalbero contenuto in T che sia un alberobello.

Discutere informalmente la correttezza dell'algoritmo proposto e la sua complessità computazionale.

Per esempio, nell'albero seguente, i nodi in azzurro formano un alberobello di dimensione 9, valore che deve essere restituito dall'algoritmo.



A3 – Most distant – Punti ≥ 12

Sia A una matrice booleana di dimensione $n \times n$ che rappresenta una mappa, dove una cella contenente **true** corrisponde a un tratto di terra, mentre una cella contenente **false** rappresenta un tratto di mare. Si assuma che esista almeno una cella contenente terra e almeno una cella contenente mare.

Scrivere un algoritmo

```
int mostDistant(int[][] A, int n)
```

che restituisca la distanza della cella di mare più distante dalla terraferma. Le distanze sono misurate tramite la *distanza di Manhattan*, ovvero la distanza fra le celle (r_1, c_1) e (r_2, c_2) è pari a $|r_1 - r_2| + |c_1 - c_2|$, cioè il numero minimo di mosse verticali o orizzontali che devono essere fatte per andare dalla prima cella alla seconda (o viceversa).

Discutere informalmente la correttezza dell'algoritmo proposto e la sua complessità computazionale.

Per esempio, nella matrice seguente, la cella mare centrale dista 4 dalle celle terra sugli angoli:

T	F	F	F	T
F	F	F	F	F
F	F	F	F	F
F	F	F	F	F
T	F	F	F	T

Algoritmi e Strutture Dati – Parte B – 14/07/25

Esercizio -1 Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

B1 – Livelli bonus – Punti ≥ 8

In un videogioco, il protagonista deve attraversare una sequenza di n livelli, associati a punteggi bonus/malus contenuti in un vettore P di interi positivi o negativi. Il personaggio parte dal livello 1. Quando supera un livello i , guadagna $P[i]$ punti e può scegliere se passare al livello $i + 1$ oppure al livello $i + 2$. Il gioco termina quando viene superato il livello n . In altre parole, il personaggio deve partire dal livello 1 e superare il livello n per terminare il gioco correttamente. Scrivere un algoritmo

```
int maxPoints(int[] P, int n)
```

che restituisca il punteggio massimo che il personaggio può accumulare per terminare il gioco.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Esempi:

- Se $P = [10, 20, -5, 30, -10, 25]$, una possibile strategia ottima è superare i livelli 1, 2, 4, 6, ottenendo $10 + 20 + 30 + 25 = 85$ punti.
- Se $P = [10, -5, -10, -30, 30, 20]$, una possibile strategia ottima è superare i livelli 1, 3, 5, 6, ottenendo $10 - 10 + 30 + 20 = 50$ punti.

B2 – Massima espressività – Punti ≥ 10

Si consideri un'espressione così rappresentata:

$$V_1 \text{ op}_1 V_2 \text{ op}_2 V_3 \dots V_{n-1} \text{ op}_{n-1} V_n$$

dove V_i sono valori interi positivi e op_i sono operatori nell'insieme $\{+, \times\}$.

Scrivere un algoritmo

```
int maxResults(int[] V, item[] op, int n)
```

dove V è il vettore dei valori di dimensione n e op è il vettore degli operandi, che restituisca il massimo valore ottenibile da una qualunque parentesizzazione dell'espressione.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Per esempio, l'espressione $5 + 8 \times 3$ può essere parentesizzata in due modi: come $(5 + 8) \times 3 = 39$ oppure come $5 + (8 \times 3) = 29$. Quindi l'algoritmo deve restituire 39.

B3 – All paths – Punti ≥ 12

Sia data una matrice A di dimensione $n \times n$, in cui ogni cella può contenere uno dei seguenti valori:

- 1: la *cella di partenza* (esattamente una);
- 2: la *cella di arrivo* (esattamente una);
- 0: una *cella vuota*, su cui si può camminare;
- -1: un *ostacolo*, che non può essere attraversato.

Il movimento è consentito nelle 4 direzioni alto, basso, destra, sinistra.

Scrivere un algoritmo

```
printAllPaths(int[][] A, int n)
```

che stampi *tutti* i percorsi validi dalla cella di partenza (valore 1) alla cella di arrivo (valore 2), tali che:

- ogni cella vuota sia visitata *esattamente una volta*;
- non siano visitati ostacoli;
- la cella di arrivo sia raggiunta *solo dopo* aver visitato tutte le celle libere;

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

La figura di esempio mostra una matrice di esempio e un possibile percorso che può essere rappresentato come $(1,1), (2,1), (3,1), (4,1), (4,2), (3,2), (2,2), (1,2), (1,3), (1,4), (2,4), (2,3), (3,3), (3,4), (4,4)$

1	0	0	0
0	0	0	0
0	0	0	0
0	0	-1	2